

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE: CSA11

Name : A K Selva Prabhu

Reg. no : 192421224

1. Smart Banking System

(a) Explain the following object-oriented concepts

Class:

A class is a blueprint that defines the properties and methods of an object. Example: *BankAccount*.

Object:

An object is an instance of a class. Example: *Customer1* is an object of the *Customer* class.

Message Passing:

Objects communicate by sending messages (calling methods). Example: Customer sends a request to transfer money.

Encapsulation:

Encapsulation hides data and allows access only through methods. This protects account information from unauthorized access.

Polymorphism:

Polymorphism allows one method to perform different tasks based on the object. Example: Different account types calculate interest differently.

(b) Object Relationships

Association:

A relationship where two objects work together. Example: Customer and Bank Account.

Aggregation:

A weak "has-a" relationship where objects can exist independently. Example: Bank has many Customers.

Composition:

A strong "has-a" relationship where one object depends on another. Example: Account cannot exist without the Bank.

(c) How these concepts improve software

- Promotes code reuse.
 - Makes software easier to maintain.
 - Improves security through encapsulation.
 - Makes adding new features easier.
 - Reduces duplicate code.
-

2. Smart Healthcare Management System

(a) Object-Oriented Design Principles

Abstraction:

Shows only important details and hides unnecessary information.

Encapsulation:

Protects patient data by restricting direct access.

Modularity:

Divides the system into smaller independent modules such as Patient, Doctor, Appointment, and Billing.

(b) SOLID Principles

Liskov Substitution Principle (LSP):

A child class should be usable wherever its parent class is used without affecting program correctness.

Interface Segregation Principle (ISP):

Classes should not be forced to implement methods they do not need. Use small, specific interfaces.

(c) Benefits

- Improves software quality.
 - Makes maintenance easier.
 - Increases flexibility.
 - Encourages reusable code.
 - Simplifies future enhancements.
-

3. Online Food Delivery System

(a) SDLC Phases

1. Planning

2. Requirement Analysis
3. Design
4. Development
5. Testing
6. Maintenance

(b) Purpose of Each Phase

Planning: Defines project goals, budget, and schedule.

Requirement Analysis: Collects customer needs.

Design: Creates system architecture, database, and UI design.

Development: Developers write the program code.

Testing: Finds and fixes software bugs.

Maintenance: Updates and improves the software after release.

(c) Importance of SDLC

- Improves software quality.
 - Reduces development risks.
 - Delivers reliable software.
 - Helps complete the project on time.
 - Makes maintenance easier.
-

4. Smart Agriculture Monitoring System

(a) SDLC Models

Prototype Model:

A sample version is built to collect user feedback before final development.

Spiral Model:

Development is carried out in cycles with continuous risk analysis.

Agile Model:

Software is developed in small iterations with frequent customer feedback.

DevOps Model:

Combines development and operations for continuous integration and deployment.

(b) Prototype vs Agile

Prototype Model

Builds an early sample

Focuses on gathering requirements

Suitable for unclear requirements

Agile Model

Builds software in iterations

Focuses on continuous delivery

Suitable for changing requirements

(c) Recommendation

The **Agile Model** is the best choice because customer requirements change regularly. Agile supports continuous feedback, quick updates, and faster delivery.

5. Smart Transport Management System

(a) Importance of UML

UML (Unified Modeling Language) is used to visually represent software design. It helps developers understand, design, and communicate system structure and behavior.

(b) UML Diagrams

State Diagram:

Shows different states of an object and how it changes.

Component Diagram:

Shows software components and their relationships.

Deployment Diagram:

Shows hardware devices and software deployment.

Package Diagram:

Shows how classes are organized into packages.

(c) Benefits of UML

- Improves communication among developers and clients.
 - Makes software easier to understand.
 - Helps detect design issues early.
 - Supports better documentation.
 - Simplifies maintenance.
-

6. Smart Inventory Management System

(a) Modular Software Design

Modular software design divides a large system into smaller independent modules. Each module performs a specific function and can be developed and tested separately.

(b) Procedural Programming vs Object-Oriented Programming

Procedural Programming	Object-Oriented Programming
-------------------------------	------------------------------------

Focuses on functions	Focuses on objects
Data is less secure	Data is protected using encapsulation
Less reusable	Highly reusable
Difficult to maintain	Easier to maintain

(c) Benefits of Modularity

- Improves scalability.
- Makes maintenance easier.
- Increases code reusability.
- Simplifies testing.
- Reduces development time.